

Report Progetto: Web Scraper & LLM Evaluation

Alessandro Angeletti
2063329

Gennaro Bernardo
2103184

David Gancino Oyos
2118949

1 Obiettivo del progetto

Il presente progetto ha come obiettivo lo sviluppo di un parser per l'estrazione di contenuti da pagine web, affiancato da un sistema di valutazione automatizzato basato su un Large Language Model (LLM) locale. Tramite API REST o un'interfaccia web dedicata, l'utente può inserire un URL, parsare la risorsa, consultare il database, gestire gli elementi salvati e visualizzare le statistiche aggregate in tempo reale. Le metriche di valutazione includono sia algoritmi classici di NLP per il confronto testuale, sia il giudizio qualitativo (LLM-as-a-Judge) rispetto a un Gold Standard di riferimento.

2 Architettura del Sistema e Organizzazione del Codice

L'infrastruttura è basata su un'architettura a microservizi containerizzati (gestiti tramite `docker-compose.yml`), suddivisa in quattro componenti principali: **frontend**, **backend**, **Ollama** e **MariaDB**.

Il **frontend** espone un'interfaccia grafica (implementata tramite template *Jinja2*) in ascolto sulla porta 8004. Questa si affianca alla documentazione generata da `/docs` (Swagger UI) e permette un'interazione user-friendly con i servizi del sistema.

Il **backend**, sviluppato in FastAPI e in ascolto sulla porta 8003, gestisce la logica di business. All'avvio, esegue in automatico il *seeding* del database, popolando la tabella `gold_standard` con i dati forniti nella directory `gs_data` (contenente 10 risorse pre-etichettate per ciascun dominio supportato, definito in `domains.json`). Inoltre, per abbattere i tempi di latenza, il backend avvia la pre-computazione asincrona delle valutazioni LLM popolando in background la tabella `evaluations`.

3 Metriche di Valutazione del Parser

L'efficacia del processo di scraping viene misurata confrontando il testo estratto dal parser con la rispettiva istanza nel `gold_standard`. La valutazione si articola su due livelli:

• Token Level Eval:

Questa metrica tokenizza i testi e ne calcola l'intersezione a livello di singole parole (bag-of-words), restituendo:

- **Precision**: la percentuale di token estratti correttamente rispetto al totale dei token parsati.
- **Recall**: la porzione di token corretti estratti rispetto al totale atteso nel Gold Standard.
- **F1-Score**: la media armonica tra Precision e Recall.

• Sequence Similarity Eval:

Un approccio basato sull'allineamento delle sequenze che valuta l'ordine strutturale dei caratteri:

- **Similarity Ratio**: un indice complessivo di somiglianza che penalizza refusi, spaziature anomale e inversioni.
- **Longest Contiguous Match**: la sequenza più lunga di caratteri perfettamente identici tra i due testi.
- **Perfect Match**: un valore booleano che attesta l'identità assoluta dei documenti.

4 Valutazione LLM-as-a-Judge

Per la valutazione semantica e qualitativa (Judge) è stato impiegato il modello **LLaMA 3.2 (3B)**. La scelta è ricaduta su questo LLM in quanto offre un eccellente compromesso tra la qualità dell'inferenza logica e i requisiti hardware per l'esecuzione in locale.

L'interazione con il modello avviene mediante il seguente approccio strutturato:

```
prompt = f"""Devi confrontare due testi tra loro. Rispondi SOLO ed ESCLUSIVAMENTE in JSON valido con 3 chiavi:
"model_name": "{MODEL}",
"judge_score": (intero tra 0 e 5),
"judge_feedback": (breve spiegazione).
T1: {dati.parsed_text[:400]}
T2: {dati.gold_text[:400]}"""

response = ollama_client.generate(
    model=MODEL,
    prompt=prompt,
    format="json",
    options={"temperature": 0.0}
)
```

Configurazione e Greedy Decoding: L'utilizzo del parametro `temperature = 0.0` è di vitale importanza. Dal punto di vista algoritmico questa impostazione cerca di rendere più deterministico l'LLM, con risposte sempre coerenti, senza improvvisare. Ciò evita il rischio di "allucinazioni", garantendo che l'output rispetti invariabilmente il formato JSON atteso e che la valutazione dei medesimi testi in input produca sistematicamente lo stesso punteggio.

5 Organizzazione del Database e Concorrenza

Il database relazionale (MariaDB), oltre a ospitare le risorse web e il Gold Standard, fa affidamento sulla tabella `evaluations`, strutturata come segue:

- `url` (Primary Key, VARCHAR)
- `precision_val`, `recall_val`, `f1_val` (FLOAT)
- `seq_ratio`, `seq_match` (FLOAT)
- `seq_perfect` (BOOLEAN)
- `judge_score` (INT)

Cooperative Processing e Gestione della Concorrenza:

L'architettura implementa un pattern di elaborazione cooperativa. Al fine di non bloccare le richieste API dell'utente, un thread isolato in background elabora progressivamente le risorse prive di valutazione e interroga il servizio Ollama.

Tuttavia, qualora un client interroghi direttamente l'endpoint di valutazione (`/full_gs_eval`) per record non ancora processati, l'API prenderà in carico la computazione (Lazy Evaluation). Per prevenire collisioni (*Race Condition*) qualora i due thread tentino di salvare l'esito per lo stesso URL nel medesimo istante, le scritture sul database sono protette dalla clausola SQL `ON DUPLICATE KEY UPDATE` (pattern *Upsert*). Questo approccio garantisce l'atomicità dell'operazione: se il record esiste già, i dati vengono sovrascritti pacificamente evitando fatali errori del motore DB ed eliminando qualsiasi rischio di deadlock.

6 Ottimizzazioni e Analisi delle Prestazioni

I benefici di questa architettura si misurano nell'abbattimento della latenza. Al primissimo avvio del container (*Cold Start*), l'elaborazione sincrona causerebbe colli di bottiglia dovuti al tempo di caricamento dei pesi del modello in RAM. Grazie alla memorizzazione persistente nella tabella `evaluations`, le successive esecuzioni della suite di testing risolvono la rotta di aggregazione in frazioni di secondo, operando semplici `SELECT SQL` invece di generare nuove code di computazione IA, notando una notevole diminuzione di tempo di esecuzione.